

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 5

The Linux File System Architecture: Inode Mechanics, Directory Link Hierarchies, Discretionary Access Control, and Virtual Kernel Pseudofilesystems

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
univ. assist.
Reiţman Patricia

Chişinău – 2026

Contents

INTRODUCTION **3**

1 THEORETICAL FILE SYSTEM MODELING AND ARCHITECTURE **3**

1.1 Virtual file system abstraction and inode semantics 4

1.2 Discretionary access control and permission masking 4

1.3 Theoretical complexity comparison 5

2 EMPIRICAL VERIFICATION AND OPERATING SYSTEM ANALYSIS **6**

2.1 File hierarchy tree structure and device nodes 7

2.2 Inode metadata and link lifecycle 8

2.3 Discretionary access control, umask evaluation, and special attributes 10

2.4 Storage subsystem layout and kernel virtual file systems 11

2.5 Advanced file system semantics and stretch investigations 13

CONCLUSIONS **15**

REFERENCES **16**

ANNEX A **17**

INTRODUCTION

The file system constitutes one of the fundamental pillars of operating system design, providing persistent storage abstractions that shield application programs from raw physical storage hardware. In modern Unix and Linux environments, the traditional maxim stating that everything is a file extends beyond disk resident data to encapsulate character streams, storage controllers, network sockets, inter-process communication channels, and real-time kernel telemetry. Understanding how the operating system organizes and mediates access to these resources requires direct inspection of the Virtual File System layer, on-disk inode architectures, directory entry mappings, and kernel access verification routines.

The objective of this laboratory work is to investigate the internal mechanisms of the Linux storage hierarchy without relying on high-level programming wrappers. By conducting controlled experiments directly against the host kernel, the study examines how file types are differentiated, how inodes decouple symbolic names from underlying physical blocks, how hard and symbolic links behave across file lifecycle operations, how Discretionary Access Control rules evaluate process privileges, and how synthetic kernel file systems present hardware topology as hierarchical directories. The scripts, empirical configurations, and automated verification suites supporting this work are maintained in the official version control repository accessible via `https://github.com/aurelian-th/os-2026-TihonAurelian`.

1 THEORETICAL FILE SYSTEM MODELING AND ARCHITECTURE

1.1 Virtual file system abstraction and inode semantics

The Linux storage architecture decouples userland file manipulation from concrete filesystem implementations through the Virtual File System (VFS) interface. As formalized in classical systems literature [1], [2], the VFS establishes four primary object primitives: the superblock, representing a mounted filesystem instance; the inode, representing a distinct metadata container for a unique storage object; the dentry, representing a cached directory entry establishing a name-to-inode relationship; and the file object, representing an active file descriptor held by an open process context.

A central principle of Unix storage design is the strict separation between file identification and file naming. An inode stores all intrinsic attributes of a file, including physical block mapping pointers, byte length, ownership credentials, access permission bitmasks, timestamps, and the reference counter tracking active hard links. Notably, the filename itself is completely absent from the inode structure. Instead, names reside strictly within directory payload blocks as arrays of directory entries. A directory is essentially a special file whose contents map arbitrary string identifiers to numeric inode indexes.

Storage allocation on block-backed filesystems operates in discrete allocation units known as blocks, typically configured to 4096 bytes. The nominal size of a file represents the actual count of readable bytes from offset zero to the end-of-file marker. Conversely, the disk space consumed reflects the aggregate volume of physical allocation blocks committed to that inode. This separation enables the creation of sparse files, wherein large ranges of zero-filled data do not trigger physical block allocations. The relationship between nominal byte size S_{nominal}, block allocation size B_{size}, and committed disk space S_{disk} is governed by the number of allocated physical sectors:

$$S_{\text{disk}} = N_{\text{blocks}} \cdot B_{\text{size}} = \left(\sum_{i=1}^{k} \mathbf{1}_{\{\text{allocated}\}}(b_i) \right) \cdot B_{\text{size}} \tag{1.1}$$

where b_i denotes the i-th logical block index of the file, $\mathbf{1}$ represents the allocation indicator function, and B_{size} corresponds to the filesystem block size. When unallocated blocks are accessed, the filesystem driver synthesizes buffers filled with null bytes without reading from physical media.

1.2 Discretionary access control and permission masking

Security enforcement in standard Linux filesystems relies on Discretionary Access Control (DAC), governed by POSIX permission bits stored inside the inode mode field. The permission mask assigns read

(r), write (w), and execute (x) rights across three distinct identity domains: the file owner user (u), the owning user group (g), and all remaining users (o).

When an executing process attempts an operation on an existing inode, the kernel evaluates permissions using a strict sequential decision ladder:

a) if the effective user identifier (EUID) of the process matches the owner UID stored in the inode, the kernel checks exclusively the user permission bits;

b) if the EUID does not match the file owner, but the effective group identifier (EGID) or any supplementary group ID matches the inode GID, the kernel checks exclusively the group permission bits;

c) if neither UID nor GID matches, the kernel checks exclusively the other permission bits.

Crucially, this logic implies that if the process UID matches the file owner, and the user permission bits deny access, the kernel returns an immediate access error (*EACCES*) without evaluating group or world permissions, even if those secondary domains grant broader privileges.

Process file creation permissions are modulated dynamically by the process creation mask (*umask*). When a process issues a file creation call such as `open()` or `creat()`, it supplies an initial base mode (0666 for regular files, 0777 for directories). The kernel derives the physical on-disk mode M_{eff} via bitwise negation and conjunction:

$$M_{\text{eff}} = M_{\text{base}} \wedge \sim U_{\text{mask}} \tag{1.2}$$

where M_{base} denotes the requested base mode, and U_{mask} represents the active creation mask of the calling process environment.

1.3 Theoretical complexity comparison

The performance of filesystem operations varies substantially depending on whether the task involves resolving human-readable string paths, traversing directory tables, indexing raw disk blocks, or looking up in-memory inode caches. Table 1.1 provides an asymptotic complexity comparison across fundamental storage operations implemented in contemporary Linux kernels [3], [4].

Table 1.1 – Asymptotic time and space complexities of filesystem operational primitives

Filesystem Operation	**Time (Average)**	**Time (Worst)**	**Space Complexity**
Direct Inode Lookup by Index	$O(1)$	$O(1)$	$O(1)$
Linear Directory Entry Scan	$O(N)$	$O(N)$	$O(1)$
Ext4 HTree B-Tree Path Search	$O(\log N)$	$O(\log N)$	$O(\log N)$
Multi-Component Path Resolution	$O(D \cdot \log N)$	$O(D \cdot N)$	$O(D)$
Sparse Block Allocation	$O(1)$	$O(1)$	$O(1)$
Hard Link Unlink Operation	$O(1)$	$O(B)$	$O(1)$

Direct inode lookup by numeric identifier executes in constant time because inodes reside at fixed block offsets computed arithmetically from the superblock table descriptor. Conversely, resolving names within classical flat directory structures scales linearly with the entry count N, whereas modern filesystems such as ext4 employ indexed HTree structures that achieve logarithmic search complexity. In Table 1.1, parameter D denotes the path depth (number of directory components), and B represents the number of physical data blocks deallocated when the final hard link counter decrements to zero.

2 EMPIRICAL VERIFICATION AND OPERATING SYSTEM ANALYSIS

2.1 File hierarchy tree structure and device nodes

The empirical investigation commenced by inspecting the root directory structure and the differentiation of file types across the system hierarchy. In Linux, file types are distinguished by the leading character in the output of the `ls -l` command. Directory entries represent containers (*d*), regular files store arbitrary data streams (−), symbolic links store pathname strings (*l*), character special devices handle unbuffered stream interactions (*c*), and block devices interface with sector-addressable media (*b*).

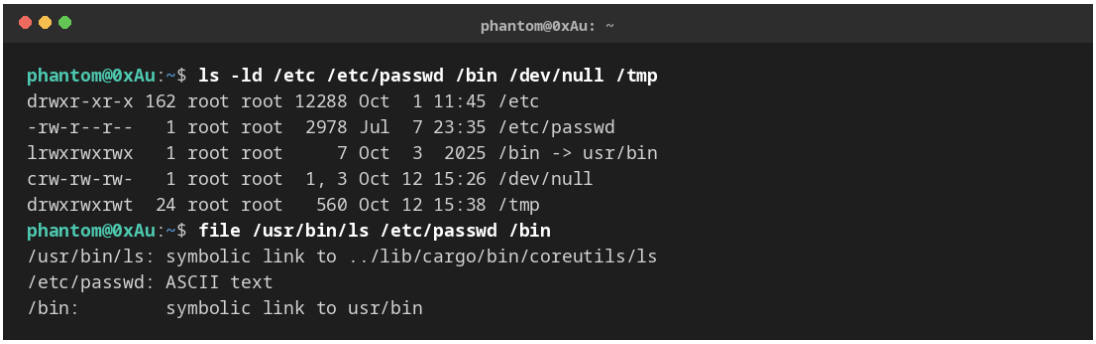


Figure 2.1 – Inspection of file type indicators and symbolic link resolution across system paths

According to Figure 2.1, the path `/etc` is identified as a directory (*d*), `/etc/passwd` as a regular file (−), `/dev/null` as a character special device (*c*), and `/tmp` as a directory configured with the restricted deletion sticky bit (*t*). Furthermore, the command revealed that `/bin` is not an independent physical directory, but a symbolic link pointing to `usr/bin`. This design reflects the UsrMerge architectural transition adopted by modern Linux distributions, consolidating executable binaries into a unified hierarchy under `/usr` while maintaining legacy compatibility through symbolic links.

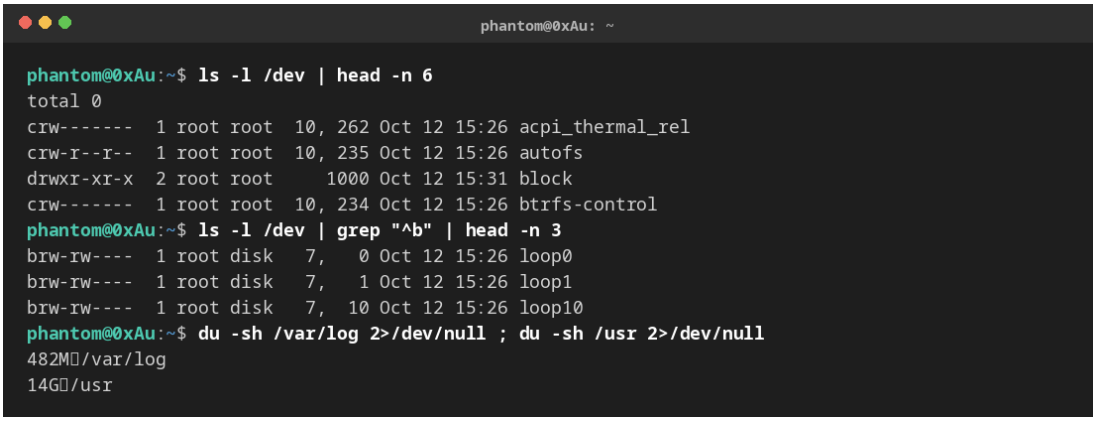


Figure 2.2 – Device node identification in the dev filesystem and storage volume consumption

The differentiation between character and block devices was validated by examining `/dev`, as doc-

umented in Figure 2.2. The device `/dev/null` operates as a character device with major device number 1 and minor number 3, discarding all inbound writes and returning an immediate end-of-file signal on reads. In contrast, `/dev/loop0` represents a block device (major 7, minor 0) capable of hosting structured filesystems. Figure 2.2 also illustrates disk consumption measurements, demonstrating that the system binary and library tree `/usr` occupies 14 GB of storage, whereas the runtime log repository `/var/log` occupies 482 MB, indicating that static system assets exceed dynamic operational logs by approximately a factor of 29.

2.2 Inode metadata and link lifecycle

To examine the relationship between filenames and on-disk inode structures, a controlled regular file named `notes.txt` was instantiated and inspected using the `stat` utility. The resulting output captured the unique numeric inode identifier, file size, block allocations, and link count.

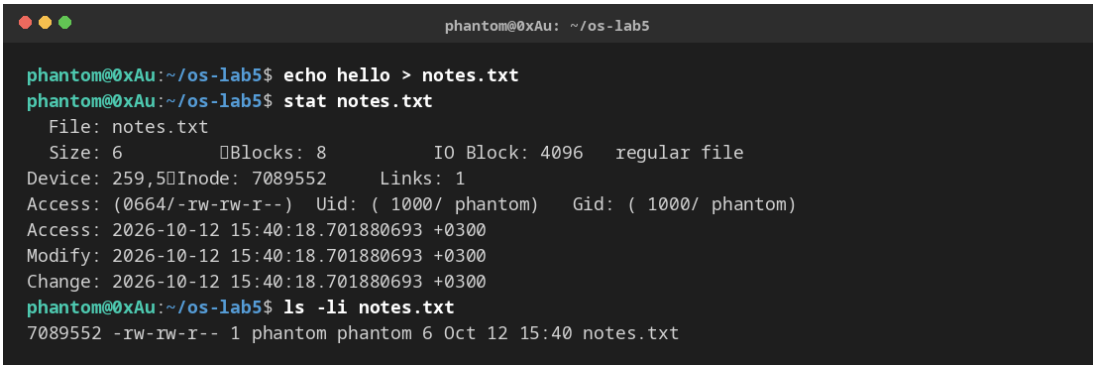


Figure 2.3 – Inode metadata extraction and sector allocation details via the stat utility

As shown in Figure 2.3, the file `notes.txt` was assigned inode number 7089552 with a link count of 1. Although the payload comprises exactly 6 bytes of ASCII text, the system allocated 8 filesystem blocks (4096 bytes of storage), confirming that physical media commits storage in discrete allocation blocks rather than arbitrary byte slices.

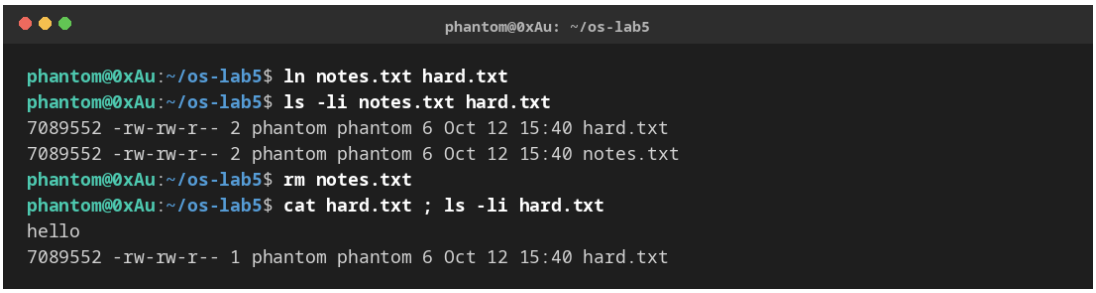


Figure 2.4 – Hard link generation, link counter progression, and data survival following unlinking

The mechanics of hard links were evaluated by creating a second directory entry `hard.txt` referencing the same target. As illustrated in Figure 2.4, both `notes.txt` and `hard.txt` share the identical

inode index 7089552, and the link counter incremented to 2. Following the removal of `notes.txt` via the `rm` command, the file contents remained fully readable through `hard.txt`, and the inode link count reverted to 1. This experiment proves that the Unix `rm` utility executes an `unlink()` system call rather than an immediate data destruction routine. The inode and its physical data blocks remain intact as long as at least one directory reference persists.

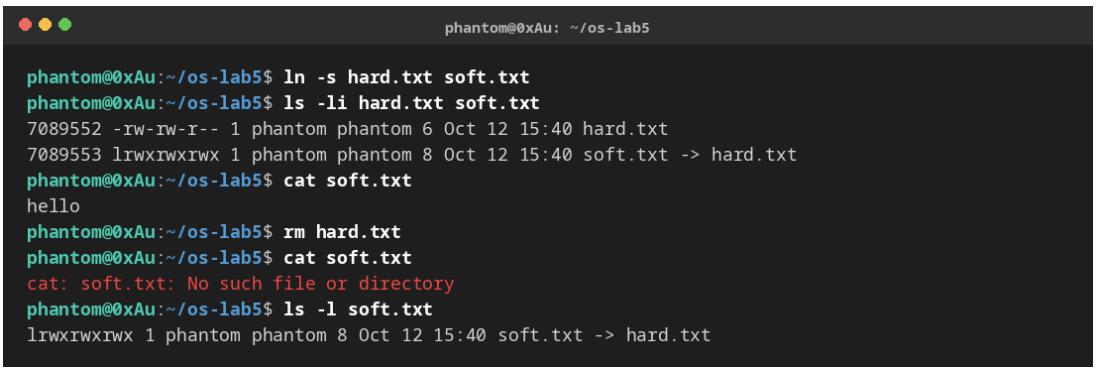


Figure 2.5 – Symbolic link creation, target unlinking, and dangling link error behavior

Conversely, symbolic links operate through an entirely distinct mechanism. As demonstrated in Figure 2.5, creating a symbolic link `soft.txt` produced a new inode (7089553) with an independent link count of 1. The size of `soft.txt` was reported as exactly 8 bytes, matching the character length of the target string `"hard.txt"`. When `hard.txt` was removed, attempting to read `soft.txt` failed with a `No such file or directory` error. Unlike hard links, symbolic links do not increment the target inode link count; when the referenced target is unlinked, the symbolic link persists as a dangling pointer.

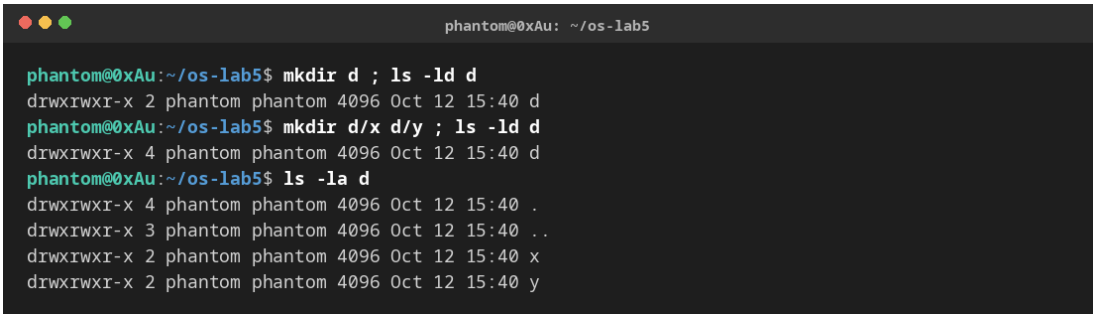


Figure 2.6 – Directory link count progression across recursive child creation

Directory link counts follow structured mathematical invariants. According to Figure 2.6, a newly created directory `d` begins with a link count of 2: one link originating from its name in the parent directory, and one internal self-referencing hard link (`.`). When two subdirectories (`d/x` and `d/y`) were instantiated within `d`, the link count of `d` increased to 4. Each subordinate directory contains a parent reference (`..`) pointing back to `d`, proving that the link count of any directory strictly satisfies $N_{\text{links}} = 2 + K$, where K represents the quantity of immediate child subdirectories.

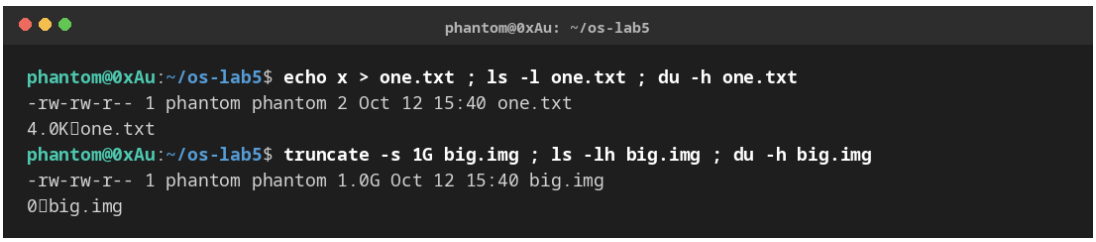


Figure 2.7 – Comparison of physical disk consumption between small files and sparse images

The distinction between nominal file size and physical storage allocation was confirmed through the sparse file test presented in Figure 2.7. A standard 2-byte file consumed 4.0 KB of disk space due to block-granularity allocation. Conversely, a 1 GB file created using `truncate -s 1G big.img` registered a nominal length of 1.0 GB in `ls -lh`, yet occupied 0 KB of physical storage in `du -h`. The kernel updated the file size field in the inode without assigning physical block pointers, demonstrating how sparse storage files conserve actual drive capacity.

2.3 Discretionary access control, umask evaluation, and special attributes

Access control behavior was analyzed by configuring file permission masks and testing process execution rights. A target file was restricted using octal modes and subjected to read attempts under varying privilege settings.

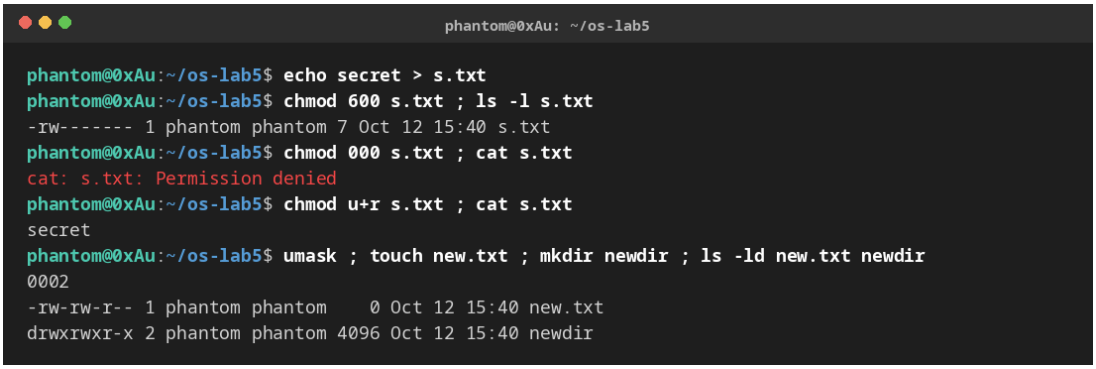


Figure 2.8 – Evaluation of DAC ownership evaluation precedence and umask mode masking

As documented in Figure 2.8, setting permissions to `chmod 000` prevented the owning user from reading the file, resulting in an immediate permission denial. This confirms that the Linux DAC algorithm inspects user bits first when the calling process EUID matches the file owner. Ownership grants permission modification rights through `chmod()`, but does not bypass DAC enforcement. Figure 2.8 also shows that with a system `umask` of `0002`, newly created files receive mode `0664` (`-rw-rw-r-`) and new directories receive `0775` (`drwxrwxr-x`), exactly adhering to Equation 1.2.

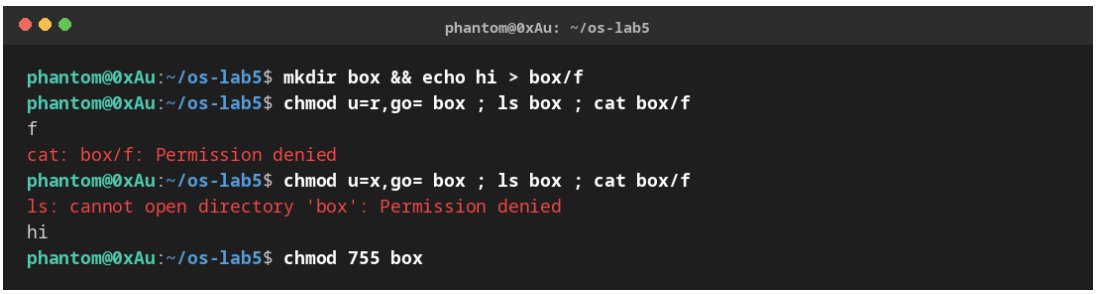


Figure 2.9 – Behavioral difference between directory read and directory search execute permissions

The semantic difference between directory read and execute permissions was isolated using the `box` directory experiment shown in Figure 2.9. When configured with read-only permission (`chmod u=r,go= box`), the user could list the entry names with `ls box`, but could not read the contents of child file `box/f`. Directory read allows enumerating directory table entries via `getdents64()`, but resolving child inodes requires the execute search bit. Conversely, when configured with execute-only permission (`chmod u=x,go= box`), `ls box` failed with permission denied because entry reading was prohibited, yet reading `box/f` succeeded directly because inode path traversal was permitted.



Figure 2.10 – Inspection of SetUID bit on passwd, sticky bit on tmp, and shadow file restriction

Special permission bits were examined in Figure 2.10. The system utility `/usr/bin/passwd` possesses the SetUID bit (`-rwsr-xr-x`), which causes the process to execute with the effective privileges of the file owner (`root`). This privilege elevation is strictly necessary because modifying user credentials requires writing to `/etc/shadow`, which is restricted to mode `0640` and owned by `root:shadow`. Figure 2.10 also shows the sticky bit on `/tmp` (`drwxrwxrwt`), ensuring that in world-writable shared directories, unprivileged users cannot delete or rename files belonging to others.

2.4 Storage subsystem layout and kernel virtual file systems

The physical storage layout and virtual kernel interfaces were examined using block query utilities and virtual filesystem exploration.

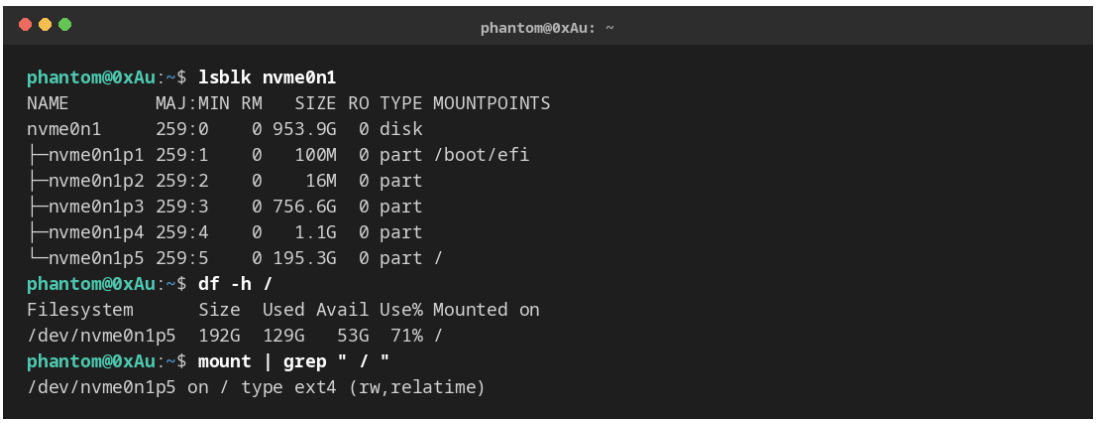


Figure 2.11 – Storage drive partition topology, root mount status, and capacity metrics

Figure 2.11 illustrates the storage topology of the host machine. The primary NVMe solid-state drive `nvme0n1` (953.9 GB) is partitioned into multiple logical regions, with partition `nvme0n1p5` (195.3 GB) formatted as an ext4 filesystem mounted at the root mount point (/). Filesystem metrics indicate that 129 GB of storage is currently in use, representing 71% capacity utilization with 53 GB of available headroom.

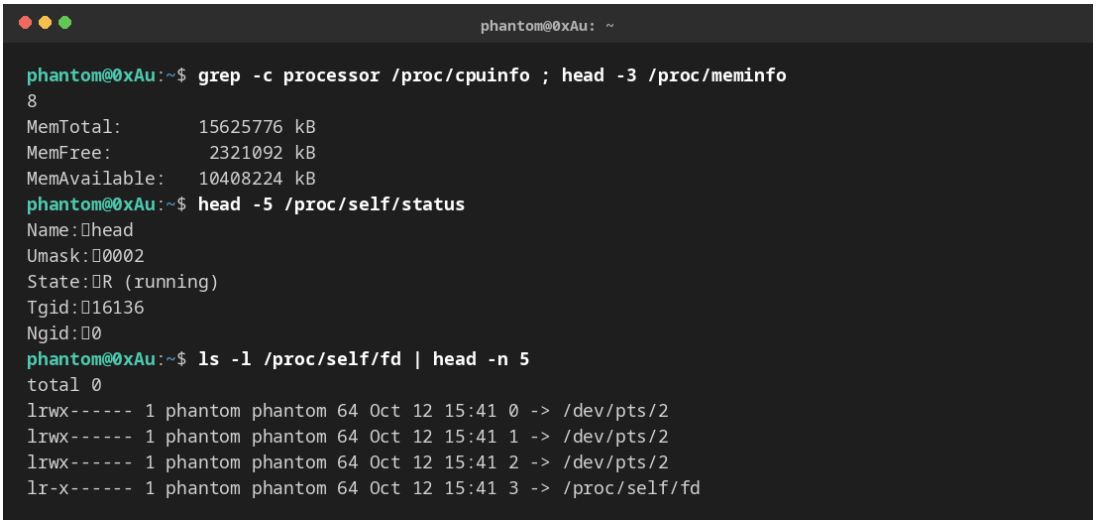


Figure 2.12 – Hardware metrics and process descriptor state exposed via the proc virtual filesystem

Kernel internal states exposed via the `/proc` pseudofilesystem were inspected in Figure 2.12. The processor query revealed 8 logical CPU execution cores, while `/proc/meminfo` reported 15.6 GB of total system RAM with 10.4 GB currently available. When querying `/proc/self/status`, the process name was reported as `head`. Because `/proc/self` is a dynamic magic link evaluated at traversal time by the kernel VFS, it automatically resolves to the PID of whichever process is currently executing the system call. Furthermore, listing `/proc/self/fd` revealed standard streams 0, 1, and 2 linked to the active terminal, alongside file descriptor 3 pointing to the directory reading handle instantiated by `ls`.

2.5 Advanced file system semantics and stretch investigations

The final experimental phase evaluated advanced file system semantics, including file signature identification, system-wide SetUID audits, and deferred deallocation mechanics during open-file unlinking.

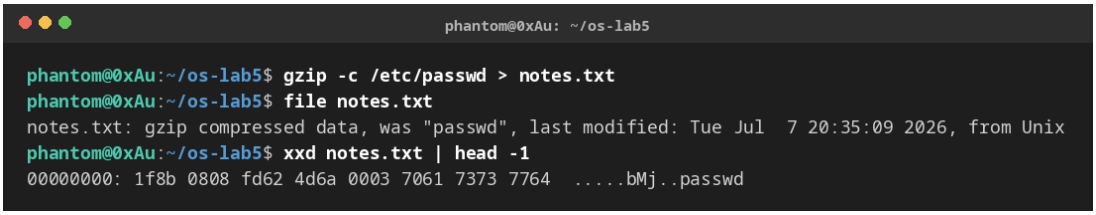


Figure 2.13 – Magic byte extraction and gzip header identification via hexadecimal inspection

According to Figure 2.13, compressing a text file produced a binary stream whose initial bytes were inspected with `xxd`. The hexadecimal dump revealed the leading bytes `1f 8b`, which constitute the standard POSIX magic signature for the gzip compressed file format. The `file` command inspects these initial byte sequences rather than filename extensions to determine the actual file type.



Figure 2.14 – System audit of SetUID binaries across the root physical partition

A security audit searching for SetUID executables was executed across the local filesystem, as shown in Figure 2.14. Binaries such as `pppd`, `chsh`, `gpasswd`, `mount`, `umount`, `pkexec`, and `passwd` were identified. Each of these utilities requires root elevation to perform restricted kernel operations, such as creating network tunnels, mounting filesystems, or modifying authentication tokens.



Figure 2.15 – Unlinking an open file descriptor and deferred block deallocation behavior

Finally, Figure 2.15 documents the behavior of a file that is unlinked while an active file descriptor remains open. By executing `exec 3< bigfile` and subsequently removing `bigfile`, the directory entry

was deleted from disk. However, inspecting `/proc/$$/fd` revealed that file descriptor 3 remained actively bound to the inode, marked by the kernel with the `(deleted)` annotation. Reading from descriptor 3 continued to yield the complete original data. This confirms that the kernel only deallocates physical disk sectors when both the inode link count drops to zero and all open file descriptors referencing that inode are closed.

CONCLUSIONS

This laboratory work provided practical validation of operating system file hierarchy concepts, inode mechanics, permission verification algorithms, and virtual pseudofilesystems through structured experimentation on a modern Linux platform. The experimental results confirmed that the Virtual File System successfully provides a uniform programming interface that abstracts diverse storage media, character devices, block devices, and runtime kernel diagnostic channels behind standard POSIX primitives. The technical objectives were fully attained, establishing a concrete understanding of how storage metadata operates independently from symbolic filenames.

Several technical insights emerged during the empirical evaluation. The hard link and symbolic link experiments clarified the exact operational semantics of the `unlink()` system call, demonstrating why data persists as long as active hard links or open file descriptors remain valid. The directory permission test cleanly separated directory entry enumeration, which depends strictly on the read bit, from directory traversal and inode resolution, which requires the execute search bit. Furthermore, testing sparse file creation demonstrated the efficiency of demand-based block allocation, highlighting the distinction between nominal file size and physical storage consumption.

From an architectural standpoint, the investigation underscored the elegance of Unix design principles. The ability of the kernel to expose process states, hardware metrics, and device drivers through unified directory interfaces simplifies systems programming and eliminates the need for ad-hoc system calls. The observed behaviors confirm theoretical expectations regarding DAC privilege checking and inode lifecycle management, providing foundational knowledge for designing resilient systems software and understanding low-level operating system architectures.

REFERENCES

1 R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2010.

2 M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, 1st ed. San Francisco, CA, USA: No Starch Press, 2010.

3 D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2005.

4 T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

ANNEX A

```bash
#!/usr/bin/env bash
# OS Laboratory Work No. 5: Automated Verification Script
set -eu

WORKDIR="${HOME}/os-lab5"
mkdir -p "${WORKDIR}" && cd "${WORKDIR}"

# Distinguishing file types and checking device nodes
ls -ld /etc /etc/passwd /bin /dev/null /tmp
file /usr/bin/ls /etc/passwd /bin
du -sh /var/log 2>/dev/null || true
du -sh /usr 2>/dev/null || true

# Inodes, hard links, and symbolic links
echo hello > notes.txt
ln notes.txt hard.txt
rm notes.txt
cat hard.txt ; ls -li hard.txt
ln -s hard.txt soft.txt
rm hard.txt
cat soft.txt 2>/dev/null || echo "Broken symbolic link"

# Directory link count progression
mkdir -p d ; ls -ld d
mkdir -p d/x d/y ; ls -ld d

# Sparse file allocation
truncate -s 1G big.img ; ls -lh big.img ; du -h big.img

# Permissions and Discretionary Access Control
echo secret > s.txt ; chmod 600 s.txt
chmod 000 s.txt ; cat s.txt 2>/dev/null || echo "Access denied"
chmod u+r s.txt ; cat s.txt

# Directory Read vs Search Permissions
mkdir -p box && echo hi > box/f
chmod u=r,go= box ; ls box ; cat box/f 2>/dev/null || echo "Traverse denied"
chmod u=x,go= box ; ls box 2>/dev/null || echo "Read denied" ; cat box/f
chmod 755 box

# Disks and Kernel Virtual Pseudofilesystems
```

```
lsblk ; df -h / ; mount | grep " / "
grep -c processor /proc/cpuinfo
head -n 3 /proc/meminfo ; head -n 5 /proc/self/status
ls -l /proc/self/fd | head -n 6 || true
```